



Artemis

SOLUTION

Observation: Let $f(x, y)$ be the number of trees below and to the left of (x, y) . Then the number of the trees in the rectangle bounded by t_1 and t_2 is

$$f(t_1.x, t_1.y) + f(t_2.x, t_2.y) - f(t_1.x, t_2.y) - f(t_1.y, t_2.x) + 1$$

if t_1 lies below and to the left of t_2 (or vice versa), and a similar formula if not.

1. Trivial algorithm. Loop over all rectangles, and loop over all trees to count those inside the rectangle.

$$O(n^3)$$

2. Use dynamic programming to compute $f(t_1.x, t_2.y)$ for every t_1, t_2 . Then evaluate all rectangles using the formulae.

$$O(n^2), \text{ but also } O(n^2) \text{ memory}$$

3. Place an outer loop t over the trees, representing one corner of a potential rectangle. To evaluate rectangles with corners at t , one only needs $f(t.x, *)$ and $f(*, t.y)$. These can be computed with DP as in algorithm (2), and requires only linear memory.

$$O(n^2)$$

4. Sort the trees from left to right, and then process them in that order. As each new tree (say t_n) is added, it is inserted into a list of current trees that is sorted vertically. From this information one can calculate $f(t.x, t_n.y)$ and $f(t_n.x, t.y)$ for every t to the left of t_n , in linear time. Then one can evaluate all rectangles with one corner at t_n . This ends up being very similar to algorithm (3).

$$O(n^2)$$

5. Algorithm (1), but with optimised counting. As a pre-process, associate a bitfield with each tree representing which trees lie below and to the right, and a similar bitfield for trees below and to the left. The trees inside a given rectangle may be found as the binary AND of two bitfields. A fast counting mechanism (such as a 16-bit lookup table) will accelerate counting.

$$O(n^3) \text{ and } O(n^2) \text{ memory, but with low constant factors}$$

Hermis

An $O(n^2)$ algorithm: Let $(x_0, y_0), \dots, (x_n, y_n)$ be the points where $(x_0, y_0) = (0, 0)$ is the starting point. We will compute $A[i, j]$ and $B[i, j]$ where $A[i, j]$ is the cost to align with the i first points and end up at (x_i, y_j) and $B[i, j]$ is the cost to align with the i first points and end up at (x_j, y_i) . We have

$$\begin{aligned} A[i+1, j] &= \min \{ A[i, j] + d[x_i, x_{i+1}], B[i, i+1] + d[y_i, y_j] \} \\ B[i+1, j] &= \min \{ B[i, j] + d[y_i, y_{i+1}], A[i, i+1] + d[x_i, x_j] \} \end{aligned}$$

The final answer is $\min_j \{A[n, j], B[j, n]\}$

Time $O(n^2)$

IOI'04: Solution of *Polygon*

Ioannis Emiris and Elias Tsigaridas

September 9, 2004

1 Minkowski decomposition

1.1 The setting

Although computing the Minkowski sum is straightforward ([2]), deciding whether a polygon is the Minkowski sum of 2 polygons is NP-complete.

There is a pseudo-polynomial time algorithm for the latter ([1]), which we now sketch. If the input is the sequence of N edges, then the input size is $O(N(\log m + \log E))$, where m is the maximum number of integer points on any edge, and E is the maximum absolute value of any coordinate defining a primitive edge vector, as defined below. The algorithm in [1] runs in $O(tNm)$ time, where $t = |P \cap \mathbb{Z}^2|$.

1.2 Solution of the IOI-04 problem *Polygon*

We call the input polygon P . Suppose that it consists of N vertices with non-negative coordinates of the form $v_i = (x_i, y_i)$, $0 \leq i \leq N-1$, given in counter-clockwise order.

Definition 1.1 *We call a vector $v = (a, b)$ primitive, if $\gcd(a, b) = 1$, where $a, b \in \mathbb{Z}_{\geq 0}$. Equivalently, v is primitive iff there are no integer points in its interior.*

The edges of P are represented by the vectors $E_i = v_i - v_{i-1} = (a_i, b_i)$, $1 \leq i \leq N$, where $a_i, b_i \in \mathbb{Z}$ and the indices are taken modulo N . Given an edge (a_i, b_i) , if $n_i = \gcd(a_i, b_i)$, then we consider the corresponding primitive edge $e_i = (\frac{a_i}{n_i}, \frac{b_i}{n_i})$. We call the sequence of vectors $\{E_i\}_{1 \leq i \leq m} = \{n_i e_i\}_{1 \leq i \leq m}$ edge sequence of the polygon, where e_i is a primitive vector.

There is a crucial observations in computing the summands A, B such that $P = A + B$.

Lemma 1.2 *Every primitive edge of P must appear as an edge either of A or of B . If the edge is not primitive, there is a third possibility: that it is the Minkowski sum of an edge of A and an edge of B , where these edges are parallel.*

Now we can deduce the following fact: A polygon is a summand of P iff its edge sequence is of the form $\{k_j e_j\}_{j \in J}$, where $J \subseteq \{1, \dots, N\}$, $0 \leq k_j \leq n_j$, $k_j \in \mathbb{Z}$ and $\sum_{j \in J} k_j e_j = (0, 0)$, i.e. the sum of the vectors that represent its edges is zero.

Below we present algorithms to find summands as required by the problem *Polygon*. In computing summand polygons, the chosen algorithm may produce a summand with one or more negative coordinates. In this case, we must shift both summands to non-negative coordinates. This is a valid operation, because shifting does not change any polygon.

1.3 Looking for a segment summand

The polygon has a segment summand iff at least one pair of the (sub)vectors of its edge sequence has a zero vector sum. Based on this fact, we present 3 algorithms of increasing speed.

Our algorithms start with finding the primitive vectors for every edge, by performing a GCD computation (at section 1.6 there are two algorithms for the GCD computation). For polygons that contain only primitive edges, this step is unnecessary.

Naive First we compute the edges of the polygon. Every edge is of the form

$$E_i = (x_{i+1} - x_i, y_{i+1} - y_i) = (a_i, b_i)$$

where the indices are taken modulo N . Next we compute all the gcd's of form $\gcd(a_i, b_i)$ and we form the edge sequence using the primitive vectors as in the preamble. This computation can be done in $O(mN)$ time, where $m = \max n_i = \max \gcd(a_i, b_i)$.

For every vector in the sequence we compute its sum with every other vector in the sequence. This can be done in $O(m^2N^2)$ total time.

Clever We compute the edge sequence as before in $O(mN)$ time. We sort the sequence with respect to their x -coordinate, in $O(mN \lg(mN))$ time.

For every vector in the sequence we test if another vector in the sequence with the opposite x -coordinate add to zero. We perform the searching by binary search, since the sequence is sorted, in $O(\lg(mN))$ time. The total time is $O(mN \lg(mN))$.

Advanced We compute the edge sequence as before in $O(mN)$ time. We insert the edges in a hash table, using as key value their x -coordinate. The insertion is performed in $O(1)$.

For every edge in the sequence we search the hash table for another edge with opposite x -coordinate that add to zero. The search is performed in $O(1)$. The total time is $O(mN)$.

1.4 Looking for a triangle summand

Naive We compute the edge sequence as before in $O(mN)$ time. We can form all the possible triplets and test if they add to zero. The total time is $O(m^3N^3)$.

Clever We compute the edge sequence as before in $O(mN)$ time and we sort it with respect to their x -coordinate in $O(mN \lg(mN))$ time.

We form all the possible sums of two vectors in $O(m^2N^2)$ time. For every such sum we search, using binary search in $O(\lg(mN))$, for another vector such that their sum is zero. The total time is $O(m^2N^2 \lg(mN))$.

Advanced We compute the edge sequence as before in $O(mN)$ time and we sort them in increasing order with respect to their x -coordinate in $O(mN \lg(mN))$ time.

For every edge in the sequence, say k , scan the sequence from the left to find i and from the right to find j , such that $E_k + E_i + E_j = 0$. Because of the sorted order we can advance either i or j according to whether the sum $E_k + E_i + E_j$ is positive or negative. The total time is $O(m^2N^2)$.

Alternatively we can insert the vectors in a hash table, using as a key their x -coordinate, and for every sum of 2 vectors we search for a vector in the hash table, in $O(1)$, such that the sum of the three is zero. Again the total time is $O(m^2N^2)$.

1.5 Looking for a quad summand

Naive We compute the edge sequence as before in $O(mN)$ time. We can form all the possible 4-tuples and test if they add to zero. The total time of the algorithm is $O(m^4N^4)$.

Clever We compute the edge sequence as before in $O(mN)$ time. We can form all the possible triplets in $O(m^3N^3)$. For every triplet we search for a vector such that the sum of the four is zero.

If we have sorted the edges with respect to their x -coordinate and we do binary searching then the total time of the algorithm is $O(m^3N^3 \lg(mN))$.

If we insert the edges in a hash table, using their x -coordinate as key, we perform the search in $O(1)$ time and so the total time of the algorithm is $O(m^3N^3)$.

Advanced We compute the edge sequence as before in $O(mN)$ time. We form all the possible sums of 2 edges, in $O(m^2N^2)$ time and we insert them in a hash table, using as key their x -coordinate. For every such sum we search the hash table for a vector such that the total sum is zero. The total time is $O(m^2N^2)$.

1.6 Computation of the gcd

We present two algorithms that compute the gcd of two integers a, b . The first algorithm is the traditional Euclid algorithm, while the second one is the Binary gcd algorithm (see [3] at chapter 4).

Algorithm 1 Euclid_gcd(a, b)

Require: $a, b \in \mathbb{Z}$

Ensure: $y = \gcd(a, b)$

if $b = 0$ **then**

 RETURN a

else

 RETURN Euclid_gcd($b, a \bmod b$)

end if

Algorithm 2 Binary_gcd(a, b)

Require: $a, b \in \mathbb{Z}$ **Ensure:** $y = \gcd(a, b)$

```
g = 1
while a is even AND b is even do
  a = a/2 (right shift)
  b = b/2
  g = 2 * g (left shift)
end while
while u > 0 do
  if a is even then
    a = a/2
  else if b is even then
    b = b/2
  else
    t = |a - b|/2
  end if
  if a < b then
    b = t
  else
    a = t
  end if
end while
RETURN g * b
```

References

- [1] S. Gao and A. Lauder, *Decomposition of polytopes and Polynomials*, Discrete and Computational Geometry 26 (2001), 501-520
- [2] L.J. Guibas and R. Seidel, *Computing Convolutions by Reciprocal Search*, Symposium of Computational Geometry, 2001
- [3] D. Knuth, *The Art of Computer Programming*, vol 2, Addison-Wesley, 3rd edition, 1998

Enumerating Hurdles

September 16, 2004

1 The problem

In comparing two DNA sequences, biologists have found out that a powerful model is based on assigning integers to genes, then compare the number of “hurdles” (to be defined below) in each sequence. This approach was used in the 1980’s to explain the genetic similarity between cabbage and turnip, in terms of evolutionary change.

Formally, we shall study permutations of the identity sequence $\sigma_I = 0\ 1\ 2\ \dots\ N\ (N+1)$, such that 0 is always the first element, $(N+1)$ is always the last element and no two consecutive integers $x\ (x+1)$ exist. The central question is to find the number and the location of certain subsequences, called *hurdles*, which are defined below.

Define a *framed interval* of a sequence σ to be an interval of the form:

$$i\ \sigma_{j+1}\ \sigma_{j+2}\ \dots\ \sigma_{j+k-1}\ i+k,$$

such that all integers between i and $i+k$ ($k \geq 1$) belong to the interval $[i+1\ \dots\ i+k-1]$. The *length* of the framed interval is the number of elements it contains including its endpoints. As a result, the length of the above framed interval is $k+1$. The *starting point* of the above framed interval is j , while the *finishing point* is $j+k$.

Hurdles are defined as framed intervals that contain no shorter framed intervals.

Your task is to write a program that, given a sequence of positive integers, finds the number of hurdles in the sequence. Your program shall also output the starting and finishing points of every hurdle.

Input: hurdles.in

The input is a permutation of the identity sequence σ_I that does not contain two consecutive integers in consecutive positions.

- The first line of the input contains the number of the elements that appear in the sequence, namely $N+2$.
- Each of the next $N+2$ lines of the input contains an integer that represents the value of the corresponding element of the sequence.

As an example, the input on sequence $\sigma_1 = 0\ 3\ 5\ 4\ 6\ 2\ 1\ 7$ would be:

Example input:
8
0
3
5
4
6
2
1
7

Output: hurdles.out

The first line of the output will contain a number that represents the number of hurdles, say M , found in the input sequence.

The next M lines will contain each a pair of numbers i, j , separated by a blank space, representing the starting and finishing points of each hurdle, subject to $i, j \in \{1, N + 2\}$. The hurdles must be sorted in increasing order of their starting points i .

The output of the above example should be:

Example output:
1
2 5

Now, let's see why the above output is correct. The entire input sequence σ_1 is a framed interval. Moreover, we have the framed interval 3 5 4 6, which can be reordered as 3 4 5 6. According to the definition, the interval 3 5 4 6 is a hurdle. This is the only hurdle in the above sequence: It has its starting point at index 2 and its finishing point at index 5.

2 Solution

Signed permutations are important in the study of DNA sequences [Ber01].

For each framed interval the following hold:

Fact 1: The starting (F_s) and finishing (F_f) point of a framed interval F are related with the following equation:

$$\text{Value}(F_f) = \text{Value}(F_s) + \text{length}(F) - 1 \quad (1)$$

Fact 2: All values in the interior points of a framed interval are distinct and cover the entire interval $[i + 1, i + k - 1]$ exactly once.

2.1 Simple

A simple algorithm is the following. For each element i of the sequence one exhaustively seeks for framed intervals of length $2 \leq l \leq (|\sigma| - i + 1)$ starting at element i , where $|\sigma| = N + 2$ is the number of elements contained in the sequence σ . For each pair (i, j) the corresponding length induces, someone inspects whether **Fact 1** holds or not. If it does, then according to **Fact 2** one must verify that all elements that lie in between are proper so that this interval is in fact a framed interval. We have $O(n^2)$ pairs (i, j) , and for each interval these pairs induce,

we require $O(n)$ time to verify that this is a framed interval whenever **Fact 1** holds. Thus, this part of the algorithm takes $O(n^3)$ time. During this part of the algorithm one keeps track of candidate hurdles starting at index i . Of course there is no need to keep track all of $O(n)$ framed intervals that might exist starting at index i , since all framed intervals that don't have minimum length can not be hurdles.

Moreover, according to our definition of hurdles, the algorithm requires another part so as to eliminate all those framed intervals that contain shorter framed intervals starting at different points. At this final part of the algorithm we have $O(n)$ candidate hurdles. For each one of them one can verify in $O(n)$ time if another candidate hurdle is contained and decide whether this candidate hurdle is really a hurdle or not. This step of the algorithm takes $O(n^2)$ time.

As a result, the entire algorithm takes $O(n^3)$ time.

2.2 Clever

Fact 3: The element with the minimum value in a hurdle is always the starting point of the hurdle.

Fact 4: The element with the maximum value in a hurdle is always the finishing point of the hurdle.

Crucial observation: An important observation for the proposed algorithm is stated by the following lemma:

Lemma 2.1 *The endpoint(s) of a hurdle can not be in the interior of another hurdle.*

Proof

Case 1: *Both endpoints of a hurdle lie in the interior of another hurdle. Then obviously according to definition, one of the above framed intervals can not be a hurdle, which is a contradiction.*

Case 2: *Only one endpoint of a hurdle lies in the interior of another hurdle.*

Suppose for the purpose of contradiction that a hurdle H_2 has its starting point k in the interior of another hurdle H_1 , a situation like the one shown in Figure 1. Then, based

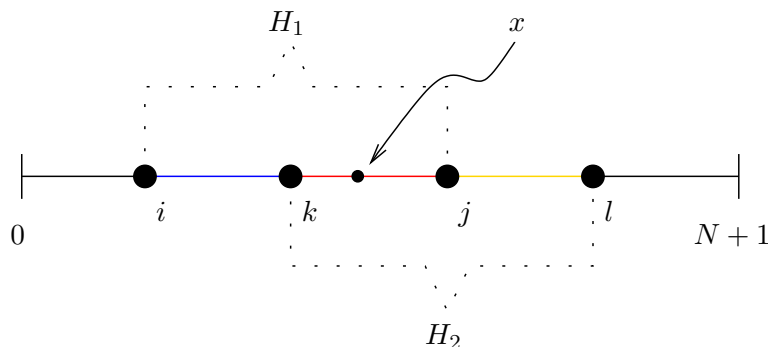


Figure 1: Proof for the second case.

on the preceding facts we have:

$$\text{Value}(i) < \text{Value}(k) < \text{Value}(j) < \text{Value}(l), \quad (2)$$

since the k^{th} element is an interior point of hurdle H_1 and j^{th} element is an interior point of hurdle H_2 . Now pick an element with index x at random such that $k < x < j$. Obviously, this element belongs to hurdle H_1 , so $\text{Value}(x) < \text{Value}(j)$. Moreover, x belongs to hurdle H_2 , so $\text{Value}(k) < \text{Value}(x)$. In other words, every element with index x , such that $k < x < j$, implies that:

$$\text{Value}(k) < \text{Value}(x) < \text{Value}(j). \quad (3)$$

Finally, there is no other element y that implies (3) outside of the interval $[k+1, j-1]$, since if it were, then either hurdle H_1 would not be a hurdle, or H_2 . As a result, all intermediate values lie in this interval (shown with red line) and as a consequence we have a framed interval. But this is a contradiction to our assumption that intervals $[i, j]$ and $[k, l]$ form two different hurdles. This completes the proof.

Note: In fact, in the above case we can prove something even stronger. Since all elements in the interval $[k+1, j-1]$ imply equation (3), all elements in the interval $[i, k]$ (blue line) imply that this region is also a framed interval since they belong to framed interval $[i, j]$. Finally, all the elements in the interval $[j, l]$ (yellow line) imply that this region is also a framed interval, since they belong to framed interval $[k, l]$.

So, a better way to handle this problem is to find a "most suitable" candidate framed interval for each element i . This can be achieved by scanning sequentially the elements of the given sequence from left to right and at the same time remembering the maximum value (max) found so far. Now, if someone finds an element x that has value $\text{Value}(x) = \text{max} + 1$ and at the same time **Fact 1** is satisfied, it is easy to see that this implies the end of a framed interval. Since we have $O(n)$ elements to check and for each one of them we have $O(n)$ elements to scan, it is straightforward that in order to find all candidate framed intervals takes $O(n^2)$ time. On the following we present the algorithm in pseudocode assuming that the elements of the sequence are given in array **SEQ**. The algorithm returns array **FRAMED**, where it is stored the length of a candidate hurdle starting at position i .

CLEVER_CANDIDATE_ENUMERATION(**SEQ**[])

```

1  for  $i \leftarrow 0$  to  $N+1$ 
2  do FRAMED[ $i$ ]  $\leftarrow$  NIL
3  for  $i \leftarrow 0$  to  $N$ 
4  do  $\text{max} \leftarrow \text{SEQ}[i]$ 
5     for  $k \leftarrow 1$  to  $N+1-i$ 
6     do if  $((\text{SEQ}[i+k] = \text{SEQ}[i] + k) \text{ AND } (\text{SEQ}[i+k] = (\text{max} + 1)))$ 
7         then FRAMED[ $i$ ]  $\leftarrow k+1$ 
8         else if  $(\text{SEQ}[i+k] < \text{SEQ}[i])$ 
9             then break
10        else if  $(\text{SEQ}[i+k] > \text{max})$ 
11            then  $\text{max} \leftarrow \text{SEQ}[i+k]$ 
12 return FRAMED
```

As in the previous algorithm, we can perform the elimination of false alerts in $O(n^2)$ time.

Based on the preceding lemma one can easily decide which of the candidate hurdles are in fact hurdles and not false alerts in linear time. One must keep track of the length of each candidate hurdle at its starting position in an array *FRAMED*. We say that a candidate hurdle is *active* if we examine elements in the interior of the hurdle.

The idea is the following. One scans sequentially the array *FRAMED*, and if he encounters a starting position of a candidate hurdle H_2 while a candidate hurdle H_1 is active, then case 1 of lemma 2.1 must apply because the candidate hurdle H_1 is a framed interval of minimum length starting at the starting position of candidate hurdle H_1 . This process requires no more than $N + 1$ queries on array *FRAMED* and as a result its running time is $O(n)$. The elimination algorithm is given with the following pseudocode:

```

LINEAR_ELIMINATION(FRAMED[])
1   $i \leftarrow 0$ 
2  while ( $i < N + 1$ )
3  do if (FRAMED[ $i$ ] = NIL)
4      then  $k \leftarrow i + 1$ 
5          while ( $k < \text{FRAMED}[i] + i - 1$ )
6              do if (FRAMED[ $k$ ] = NIL)
7                  then FRAMED[ $i$ ]  $\leftarrow$  NIL
8                  break
9                   $k \leftarrow k + 1$ 
10              $i \leftarrow k - 1$ 
11      $i \leftarrow i + 1$ 
12 return FRAMED

```

Finally, either way we choose to eliminate false alerts, this algorithm has running time $O(n^2)$.

2.3 Advanced

This problem can be solved in superlinear $O(n\alpha(n))$ [BH96] and linear ($O(n)$) [BMY01] time if someone uses graphs. These results come from one of the most exciting areas in computer science in the last decade which was pioneered by Hannenhalli and Pevzner [HP95] and have rather simple implementations.

A detailed treatment of the field that inspired this problem can be found at [Sie01].

References

- [Ber01] Anne Bergeron. A Very Elementary Presentation of the Hannenhalli-Pevzner Theory. *CPM 2001 - Lecture Notes in Computer Science*, 2089:106–117, April 19 2001.
- [BH96] Piotr Berman and Sridhar Hannenhalli. Fast Sorting by Reversal. *Proceedings of the 7th Annual Symposium on Combinatorial Pattern Matching*, pages 168–185, 1996.
- [BMY01] Bader, Moret, and Yan. A Linear-Time Algorithm for Computing Inversion Distance between Signed Permutations with an Experimental Study. In *Lecture Notes in Computer Science*, editor, *WADS: 7th Workshop on Algorithms and Data Structures*, volume 2125, pages 365–376, 2001.

- [HP95] Sridhar Hannenhalli and Pavel Pevzner. Transforming Cabbage into Turnip. *Proceedings of the 27th Annual Symposium on Theory of Computing (STOC95)*, pages 178–189, 1995.
- [Sie01] Adam C. Siepel. Exact Algorithms for the Reversal Median Problem. Master’s thesis, University of New Mexico, December 2001.
- [Sie02] Adam C. Siepel. An Algorithm to Enumerate All Sorting Reversals. *RECOMB*, pages 281–290, April 2002.

The Farmer

Let g_i be a field or a strip. Denote by n_i the number of cypresses in a field or a strip. If we denote by e_i the number of olive trees in a g_i , we have: $e_i = n_i$ if g_i is a field or $e_i = n_i - 1$ if g_i is a strip.

Consider now the following KNAPSACK problem: $\max \sum_{i=1}^{n+m} e_i x_i$, such that $\sum_{i=1}^{n+m} n_i x_i \leq Q$ and $x_i \in \{0, 1\}$, where n, m are the numbers of fields and strips respectively.

An optimum solution x_i^* , $1 \leq i \leq n + m$, of this problem consists of a subset of g_i such that the total number of their cypresses is at most Q and the total number of the included olive trees is maximized. However in general $Q' = \sum_{i=1}^{n+m} n_i x_i^* < Q$. If this is the case, then there is some g_i such that $x_i^* = 0$ and $n_i > Q - Q'$, for otherwise the optimum solution can be improved by the inclusion of g_i , a contradiction. Therefore, adding a chain of $Q - Q'$ cypresses of g_i and its $Q - Q' - 1$ olive trees to the optimum solution of the knapsack problem above, yields to an optimum solution. The KNAPSACK problem can be solved optimally in $O((n + m)Q)$ time by a Dynamic Programming algorithm.

Use Dynamic Programming:

Let $a(x, y)$ be the wasted area for a rectangle (x, y) , $1 \leq x \leq W$, $1 \leq y \leq H$. Initially, put $a(x, y) = xy$, for all (x, y) except for the ones corresponding to needed plates, e.g. $x = w_i$ and $y = h_i$, $1 \leq i \leq N$, for which we put $a(x, y) = 0$. For a plate (x, y) consider all vertical cuts $c = 1, 2, \dots, x - 1$ and all horizontal cuts $c = 1, 2, \dots, y - 1$ and chose the cut producing the minimum wasted area $a(x, y) = a(c, y) + (x - c)y$ or $a(x, c) + a(x, y - c)$ for some c .